PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026–27

GENERATIVE AI COURSE PROJECT REPORT

RESEARCHPILOT AI

An Intelligent Research Assistant with Supervisor Agent Architecture

Faculty Name: [FACULTY NAME]

Team Members

1. K. Jan Vikash	23501A0571
2. K. Harsha Vardhan	23501A0584
3. K. Sruthi	23501A0573
4. K. Yoshita Navya	23501A0581

GitHub Repository Link: [INSERT REPOSITORY URL]

Project Deployment Link: [INSERT DEPLOYMENT URL OR WRITE NOT DEPLOYED]

Deployment Platform: [INSERT PLATFORM / NOT APPLICABLE]

Signature of Faculty **Signature of HOD**

CONTENTS

CHAPTER 1: INTRODUCTION **3**

1.1 Background • 1.2 Problem Statement • 1.3 Motivation • 1.4 Objectives • 1.5 Scope

CHAPTER 2: LITERATURE SURVEY **5**

2.1 Introduction • 2.2 Review of Research • 2.3 Comparison Table • 2.4 Research Gap

CHAPTER 3: PROBLEM DEFINITION AND REQUIREMENTS **8**

3.1 Existing System • 3.2 Limitations • 3.3 Proposed System • 3.4 Functional Reqs • 3.7 Software

CHAPTER 4: SYSTEM DESIGN **10**

4.1 Overall Architecture • 4.2 Supervisor Agent • 4.4 Intent Hierarchy • 4.6 RAG • 4.10 Sequence

CHAPTER 5: IMPLEMENTATION **17**

5.1 Frontend • 5.2 Backend • 5.4 Search • 5.8 Intent Routing • 5.9 Tools • 5.10 Firewall

CHAPTER 6: TESTING AND RESULTS **20**

6.1 Introduction • 6.2 Test Cases (TC01-TC18) • 6.3 Results • 6.7 Limitations

CHAPTER 7: CONCLUSION AND FUTURE SCOPE **22**

7.1 Conclusion • 7.2 Limitations • 7.3 Future Scope

CHAPTER 8: REFERENCES **23**

APPENDIX A: IMPLEMENTATION WORKFLOW AND API RESPONSIBILITIES **24**

APPENDIX B: APPLICATION SCREENSHOT PLACEHOLDERS **25**

Note: Page numbers reflect the complete 27-page academic report structure. All technical corrections (supervisor routing priorities, diagram alignments, grounded summarization, and firewall specifications) have been fully integrated.

CHAPTER 1: INTRODUCTION

1.1 Background

Academic research requires students and researchers to discover relevant publications, understand technical contributions, compare methodologies, and identify unanswered questions. These activities often involve searching several scholarly repositories, downloading papers, reading long PDF documents, recording notes, and preparing literature reviews. When these tasks are performed manually across separate websites and documents, the process can become time-consuming and difficult to organise.

Recent advances in large language models (LLMs), information retrieval, document parsing, and vector databases provide opportunities to support this workflow. Retrieval-Augmented Generation (RAG) combines a retrieval component with language-model generation so that responses can be informed by selected source material. Tool-using language-model systems can also route a request to a suitable operation, such as search, summarisation, or document question answering.

ResearchPilot AI is a web-based academic research assistant. The supplied implementation combines a React and TypeScript frontend with a Python FastAPI backend, relational storage through SQLite and SQLAlchemy, and a persistent ChromaDB collection for document retrieval. The application supports academic paper discovery, workspace organisation, PDF upload and extraction, workspace-aware question answering, and structured research analysis.

The project uses a Supervisor Agent to classify a research request and invoke one of six existing research tools. This is best described as a partially agentic architecture: the supervisor makes a bounded routing decision and calls a selected tool, while the tools perform defined tasks. The code does not establish unrestricted autonomous planning, independent agent-to-agent collaboration, or long-term autonomous research.

1.2 Problem Statement

Students and researchers frequently need to search multiple academic sources, organise papers by topic, extract information from PDFs, and synthesise findings across publications. These tasks are commonly performed with separate search portals, PDF readers, note-taking applications, and general-purpose chat interfaces. This fragmentation makes it harder to preserve a clear connection between an analytical response and the papers that support it.

General-purpose language models may produce plausible but unsupported statements when they are not grounded in the relevant papers. A research assistant therefore needs to retrieve relevant document passages, retain source metadata, and make limitations visible when evidence is unavailable. It should also recognise the type of research task and select a suitable operation without requiring the user to navigate to a separate tool for every request.

The problem addressed by ResearchPilot AI is the design and implementation of an integrated academic research platform that combines paper discovery, workspace-based literature organisation, PDF processing, retrieval-grounded chat, specialised research tools, and evidence-oriented response checking within a single interface.

1.3 Motivation

- To reduce the effort involved in discovering and organising academic literature.
- To provide a separate workspace for each research topic or project.
- To make uploaded PDF content searchable through text extraction, chunking, embeddings, and vector retrieval.
- To help users ask questions about the papers stored in a selected workspace.
- To support structured summarisation, literature synthesis, comparison, and research-gap analysis.
- To make tool selection visible through a Supervisor Agent execution trace.
- To encourage evidence-based responses and expose cases where a claim lacks supporting material.

1.4 Objectives

- Develop a browser-based academic research application using React, TypeScript, and Vite.

- Implement backend APIs using Python and FastAPI.
- Provide user authentication and workspace-level organisation of research material.
- Search supported scholarly sources and present paper metadata and access links.
- Extract text from uploaded PDFs using PyPDF and index paper chunks in ChromaDB.
- Implement workspace RAG question answering with retrieved source passages.
- Provide six research tools: academic search, summarisation, literature review, comparison, research-gap identification, and workspace Q&A.;
- Implement a Supervisor Agent that identifies intent and routes a query to one appropriate existing tool.
- Display an execution trace containing identified intent, selected tool, routing rationale, and execution status.
- Apply evidence and citation checks to generated research responses.

1.5 Scope of the Project

The project scope covers a web application for individual academic research workflows. Users can register and sign in, create workspaces, search academic indexes, import papers into a workspace, upload PDF files, and interact with papers through chat and analysis tools. The source code includes service modules for arXiv, PubMed, Semantic Scholar, Crossref, CORE, DOAJ, IEEE, ACM, and conference proceedings sources including NeurIPS, ICML, and ICLR. OpenAlex is utilized within the discovery pipeline as an open-access URL and DOI metadata enrichment resolver rather than an initial keyword discovery adapter. Actual result availability depends on each connector and external service.

The backend includes a PDF extraction and vector-indexing path, section-aware text chunking, ChromaDB storage, and retrieval using dense and lexical/evidence signals. The embedding implementation attempts to load Sentence Transformers all-MiniLM-L6-v2; when it cannot load, the code uses deterministic mock vectors. This fallback is operationally important because mock vectors are not equivalent to semantic embeddings.

The Supervisor Agent performs bounded intent classification and tool routing. The six routed tools are defined in the source code. The application should not be represented as a fully autonomous multi-agent system. The project does not claim autonomous multi-step research planning, independent agent collaboration, or persistent long-term memory beyond the database-backed workspace and chat records.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

This chapter reviews research that informs retrieval-augmented generation, tool-using language models, academic information retrieval, and the detection of unsupported generated content. These works provide conceptual foundations for ResearchPilot AI; they are not claims that the application reproduces every research method in the cited papers.

2.2 Review of Existing Research

1. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors: Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Year: 2020. Methodology: The paper combines a parametric sequence-to-sequence generator with a non-parametric dense vector index of retrieved passages. Retrieval supplies external evidence that conditions text generation. Application to ResearchPilot AI: This is the main conceptual basis for retrieving relevant chunks from a workspace before answering research questions. Limitation: RAG quality depends on the coverage, relevance, and quality of the indexed corpus and retrieval method; retrieval does not guarantee that the generator will use evidence correctly.

2. ReAct: Synergizing Reasoning and Acting in Language Models

Authors: Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. Year: 2023. Methodology: ReAct interleaves reasoning traces and actions, enabling a language model to interact with tools or environments. Application: The work motivates the separation of intent understanding, tool selection, tool execution, and response synthesis. ResearchPilot AI implements a narrower deterministic intent-routing mechanism rather than the full ReAct loop. Limitation: Tool use can fail through incorrect tool selection, malformed arguments, or unreliable tool outputs; reasoning text alone does not establish correctness.

3. Toolformer: Language Models Can Teach Themselves to Use Tools

Authors: Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Year: 2023. Methodology: Toolformer studies how language models can learn when to call APIs and how to incorporate tool responses during language modelling. Application: It provides background for systems that connect a language model to external academic search and document-analysis capabilities. ResearchPilot AI uses a supervisor and explicit tool registry rather than claiming Toolformer-style self-supervised tool learning. Limitation: Tool availability and correctness depend on API design, data quality, permissions, and robust error handling.

4. Dense Passage Retrieval for Open-Domain Question Answering

Authors: Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Year: 2020. Methodology: Dense Passage Retrieval learns question and passage representations and retrieves passages by vector similarity. Application: This work informs the use of embedding-based retrieval over chunks extracted from research PDFs. ResearchPilot AI uses ChromaDB and a configured sentence-transformer model when available, supplemented by lexical and evidence-oriented scores. Limitation: Dense retrieval depends on embedding quality and domain fit; a model-loading fallback to deterministic mock vectors should not be interpreted as meaningful semantic retrieval.

5. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models

Authors: Potsawee Manakul, Adian Liusie, and Mark Gales. Year: 2023. Methodology: SelfCheckGPT proposes checking consistency across multiple sampled responses as a zero-resource signal for possible hallucinations. Application: It provides a research basis for treating hallucination detection as a separate verification problem after generation. ResearchPilot AI's source implements evidence/citation-oriented checks; it should not be described as implementing the SelfCheckGPT sampling method. Limitation: Consistency is only an indirect signal and may fail to detect consistently repeated false statements.

6. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions

Authors: Lei Huang and co-authors. Year: 2023. Methodology: The survey organises hallucination definitions, causes, detection approaches, and mitigation strategies for large language models. Application: It supports the design rationale for a separate evidence and citation verification stage and the need to report uncertainty when supporting passages are absent. Limitation: A survey organises the research landscape but does not itself provide a universally reliable detector.

2.3 Literature Comparison Table

Author / Work	Year	Core method	Relevance to ResearchPilot AI	Limitation
Lewis et al., RAG	2020	Retrieval + generation	Grounded workspace Q&A;	Depends on corpus and retrieval quality
Yao et al., ReAct	2023	Reasoning + actions	Conceptual basis for bounded routing	Tool outputs still need validation
Schick et al., Toolformer	2023	Learned API use	Background for tool-enabled LLMs	Not the same as explicit rule routing
Karpukhin et al., DPR	2020	Dense passage retrieval	Embedding-based chunk retrieval	Embedding quality affects relevance
Manakul et al., SelfCheckGPT	2023	Cross-sample consistency checks	Hallucination detection motivation	Not a direct evidence guarantee
Huang et al., Hallucination Survey	2023	Taxonomy and review	Verification design rationale	Survey does not solve detection

2.4 Research Gap

Prior work provides strong individual techniques for retrieval, tool use, dense passage search, and hallucination analysis. A student-oriented research workflow still needs these capabilities to be connected to practical workspace operations. Users need to move from discovery to PDF ingestion, ask questions over their own papers, and perform synthesis tasks without manually coordinating several disconnected utilities.

ResearchPilot AI addresses this integration gap through a workspace-centred application and a bounded Supervisor Agent. Its contribution is primarily system integration: academic search services, PDF parsing, persistent vector indexing, RAG chat, specialised analysis functions, and an execution trace are exposed through one application. The evidence firewall further reflects the need to distinguish generated prose from content that can be tied to retrieved paper evidence.

The implementation remains constrained by the reliability of external scholarly sources, PDF text extraction quality, embedding availability, retrieval ranking, language-model provider availability, and the strength of the evidence checks. These constraints are acknowledged rather than treated as solved research problems.

2.5 References Used in Literature Survey

- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems 33. https://arxiv.org/abs/2005.11401
- Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations. https://arxiv.org/abs/2210.03629
- Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems 36. https://arxiv.org/abs/2302.04761
- Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. https://arxiv.org/abs/2004.04906
- Manakul, P., Liusie, A., & Gales, M. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023. https://arxiv.org/abs/2303.08896

• Huang, L., et al. (2023). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. https://arxiv.org/abs/2311.05232

(Literature Survey references concluded. Proceed to Chapter 3 for problem definition and system specifications.)

CHAPTER 3: PROBLEM DEFINITION AND REQUIREMENTS

3.1 Existing System

In a conventional research workflow, users search scholarly websites independently, download papers, extract important details manually, and organise findings in notes or spreadsheets. General-purpose chat assistants may help explain concepts, but unless connected to the user's papers they may not have access to the exact document content. A researcher may also need separate tools for summarisation, comparison, and literature review.

3.2 Limitations of Existing System

- Research information is distributed across multiple repositories and applications.
- Manual paper organisation makes it difficult to maintain topic-specific collections.
- Reading and comparing many long PDFs requires significant time.
- General chat responses may not be grounded in the user's selected papers.
- Separate research utilities require repeated context entry and navigation.
- Unverified generated statements or references can reduce confidence in an AI-assisted literature review.
- Scanned or image-only PDFs may not yield extractable text through ordinary text extraction.

3.3 Proposed System

ResearchPilot AI integrates the main stages of an academic literature workflow in one web application. A user creates or selects a workspace, discovers papers through supported academic search connectors, imports relevant publications, and uploads PDFs where required. Extracted text is segmented into chunks and indexed in ChromaDB. The user can then ask workspace questions or request structured analysis.

The Supervisor Agent is the control component for research chat requests. It examines the user's wording, selects one of six registered research tools according to an explicit priority order, invokes that tool, and returns a trace describing the identified intent, selected tool, routing rationale, and execution status. This is bounded tool orchestration, not unrestricted autonomous planning.

The Hallucination Firewall is treated as an evidence-oriented validation stage. It checks whether generated research statements and citations are supported by available retrieved material. The strength of this mechanism depends on the exact verification logic and available source metadata; it cannot guarantee that every factual statement is correct.

3.4 Functional Requirements

ID	Functional requirement
FR01	The system shall allow users to register, log in, and access protected application features.
FR02	The system shall allow users to create and manage research workspaces.
FR03	The system shall search supported academic sources and display paper metadata.
FR04	The system shall allow a user to import a paper into a workspace.
FR05	The system shall accept PDF uploads and attempt text extraction using PyPDF.
FR06	The system shall divide extracted text into chunks and index them in ChromaDB.
FR07	The system shall retrieve relevant workspace passages for research questions.
FR08	The system shall provide paper summarisation, literature review, comparison, and research-gap analysis.
FR09	The Supervisor Agent shall classify a query and route it to one appropriate research tool.

ID	Functional requirement (continued)
FR10	The system shall display an execution trace for supervisor-routed requests.
FR11	The system shall expose evidence/source metadata for grounded responses where available.
FR12	The system shall handle unavailable providers, missing papers, invalid requests, and extraction errors with suitable messages.

3.5 Non-Functional Requirements

- **Usability:** clear navigation between dashboard, workspaces, search, upload, chat, and AI tools.
- **Security:** authenticated access, password hashing, token-based authorization, and workspace ownership checks.
- **Reliability:** graceful handling of failed academic connectors, unavailable LLM providers, and unsupported PDF extraction.
- **Maintainability:** modular FastAPI routers, service classes, SQLAlchemy models, and React page components.
- **Performance:** retrieval and analysis should complete within practical interactive limits; actual timings require measurement.
- **Portability:** the frontend should run in modern browsers and the backend in a supported Python environment.
- **Evidence quality:** responses should identify source material and avoid presenting unsupported content as verified.

3.6 Hardware Requirements

Component	Suggested development requirement
Processor	Modern dual-core processor or better
RAM	8 GB recommended; additional memory may be useful for local embedding models
Storage	Space for source code, uploaded PDFs, SQLite database, and ChromaDB index
Internet	Required for hosted LLM providers and remote academic search sources
Display	Standard laptop or desktop display

3.7 Software Requirements

Software / Service	Purpose
React 19, TypeScript, Vite	Frontend interface and development server
Python 3.9+ / FastAPI / Uvicorn	Backend APIs and application server
SQLite / SQLAlchemy	Relational persistence and ORM
ChromaDB	Persistent vector storage and retrieval
PyPDF	Text extraction from uploaded PDF documents
Sentence Transformers	Embedding generation when model is available (fallback: deterministic vectors)
Groq LLaMA 3.3 70B Versatile	Configured primary LLM model
Gemini / OpenRouter / Ollama	Additional provider paths configured in the LLM service, subject to settings
Git / Code editor / Browser	Development, version control, and application access

CHAPTER 4: SYSTEM DESIGN

4.1 Overall System Architecture

The architecture is divided into a browser-based presentation layer, FastAPI application layer, service layer, relational persistence layer, vector retrieval layer, and external provider layer. React components communicate with backend endpoints. The backend validates the user and workspace context, then calls the relevant service. Relational entities are stored through SQLAlchemy, while extracted paper chunks and their embeddings are stored in ChromaDB.

The LLM service is configured with Groq and the model name llama-3.3-70b-versatile. The implementation also contains provider functions for Gemini, OpenRouter, and Ollama. The provider chain may attempt alternatives depending on configuration; the AI_TOOLS_GROQ_ONLY setting can restrict the chain. Therefore, the report identifies Groq as the configured primary model and describes other providers as conditional fallbacks, not guaranteed active services.

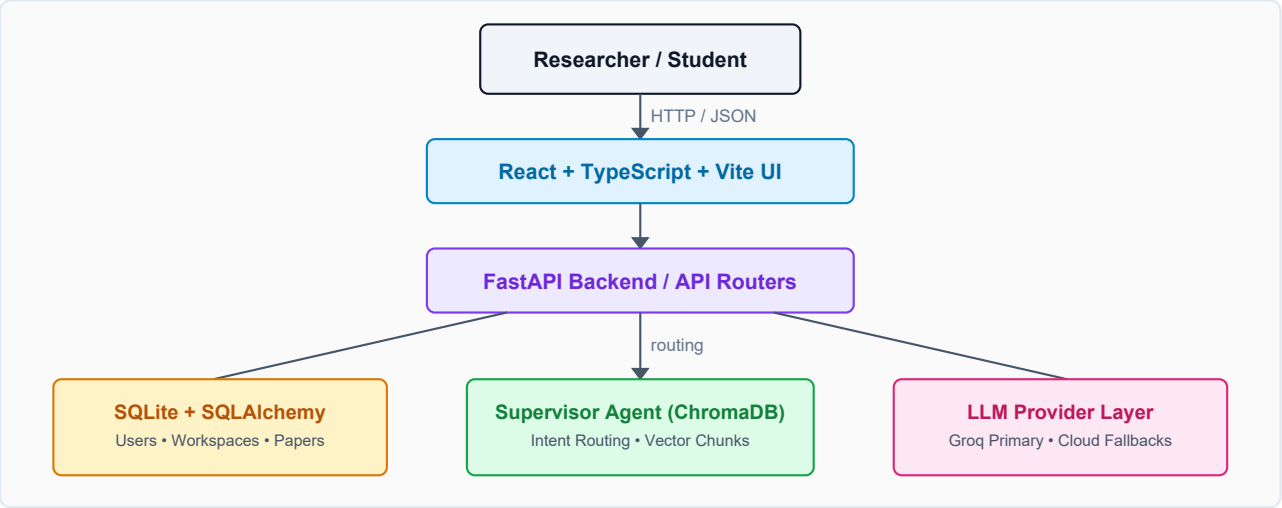


Figure 4.1: High-level architecture of ResearchPilot AI.

4.2 Supervisor Agent Architecture

The Supervisor Agent is implemented in the backend service supervisor_agent.py. It receives a research query together with workspace and session identifiers. It loads papers belonging to the active workspace, optionally narrows the paper set when selected paper identifiers are supplied, and calls the intent-classification method. The selected tool is then executed through a corresponding method.

The classifier uses ordered text-pattern rules evaluated in strict hierarchical priority: Comparison has the highest priority (Priority 1), followed by literature review (Priority 2), paper summarisation (Priority 3), research-gap identification (Priority 4), explicit academic search (Priority 5), and finally workspace question answering (Priority 6). This priority matters when a query contains more than one task-related phrase. The supervisor returns an execution banner that includes the detected intent, selected tool, reasoning text, and execution status.

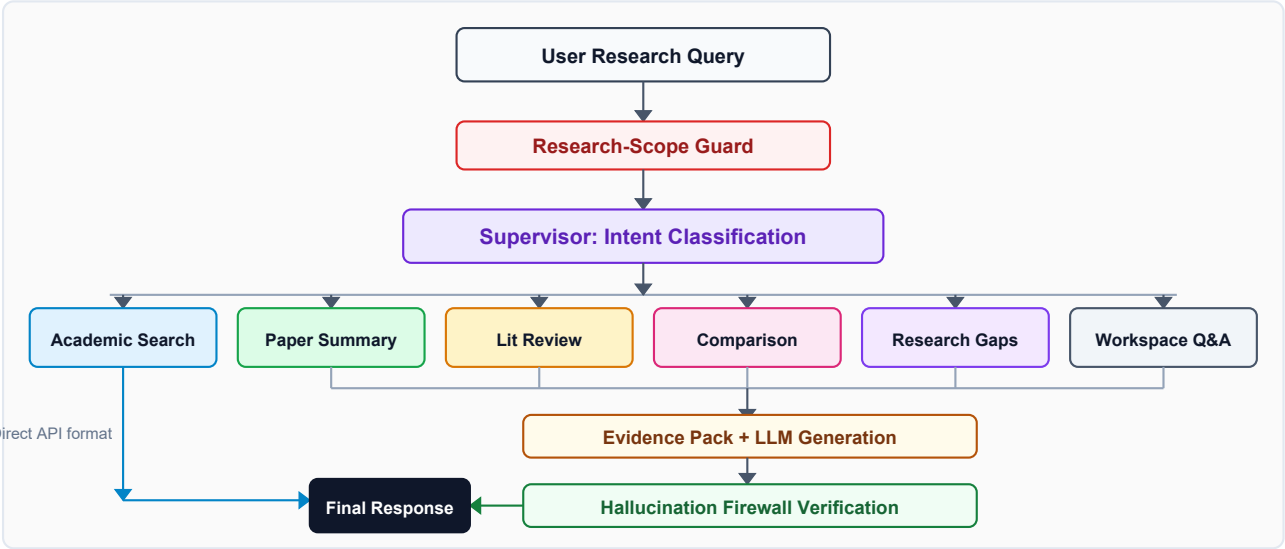


Figure 4.2: Supervisor Agent intent classification and tool routing.

4.3 Agent Roles and Responsibilities

Component	Responsibility	Output
Supervisor Agent	Classifies the user's research intention and selects one registered tool	Intent, tool selection, reasoning, execution status
Academic Paper Search	Calls the hybrid search service and formats paper metadata (bypasses LLM)	Search results and access links
Paper Summarization	Builds a paper evidence payload and requests a structured summary	Summary of objectives, methods, dataset, findings, limitations
Literature Review	Collects workspace paper evidence and synthesises themes	Cross-paper literature review
Paper Comparison	Prepares paper payloads and requests comparative analysis	Comparative matrix / narrative
Research Gap Identification	Analyses evidence from workspace papers	Potential gaps, limitations, and open questions
Workspace RAG Q&A;	Retrieves relevant chunks and generates a grounded answer	Answer with source context
Hallucination Firewall	Passively checks claim and citation support against available evidence	Retained, flagged, or qualified response with audit stats

4.4 Intent Classification and Tool Selection

The implementation defines six tool identifiers: paper_comparison, literature_review, paper_summarization, research_gap_identification, academic_paper_search, and workspace_qa. The supervisor uses regular-expression patterns and a fixed priority order rather than an unconstrained agent planner. It also extracts search terms and attempts to match a paper title mentioned by the user to a paper in the workspace using multi-tier heuristics (quotes, substrings, and word overlap).

Table 4.2 shows the routing map in exact execution order. When a user request satisfies higher-priority comparison or review patterns, those analytical tools are executed first. Specific single-paper summary directives take precedence over general limitation queries, while explicit repository search terms invoke academic discovery. Unmatched factual or conceptual questions default to conversational workspace RAG.

Priority	User intention / pattern triggers	Selected tool
1	Compare / contrast / versus / comparative matrix / side by side	Paper Comparison
2	Literature review / synthesize papers / thematic review / SOTA review	Literature Review
3	Summarize / provide summary / key contributions / brief overview	Paper Summarization
4	Explicitly identify, find, detect, uncover, or explore research gaps, open questions, and unaddressed research problems	Research Gap Identification
5	Search or discover academic papers / search arXiv / find literature	Academic Paper Search
6	Specific conceptual or factual question about workspace papers (default)	Workspace RAG Q&A;

4.5 Research Tool Execution Workflow

After selecting a tool, the supervisor calls only the corresponding execution method for that request. Academic search invokes the hybrid search service. Summarisation, review, comparison, and gap analysis build evidence payloads from selected workspace papers and pass them to the LLM service. Workspace Q&A; retrieves up to eight relevant chunks by default, or retrieves chunks from explicitly selected papers, then supplies the query, chat history, and paper metadata to the RAG generation method.

The implementation includes checks for insufficient input. Literature review and comparison require at least two papers. If no paper is available for summarisation or gap analysis, the response asks the user to upload or import papers rather than inventing an analysis. The number of papers included in some operations is bounded in code: up to six for literature review and gap analysis, and up to five for comparison.

4.6 RAG Architecture

The PDF path extracts text from uploaded files using PyPDF. The vector service detects common academic section headings and divides text into paragraph-aware chunks. The configured default chunk size is 850 characters with 160 characters of overlap. Each chunk is associated with metadata such as paper identifier, workspace identifier, chunk index, title, and section. Embeddings are stored in a persistent ChromaDB collection named research_papers.

The embedding helper attempts to load sentence-transformers/all-MiniLM-L6-v2. If the package or model cannot be loaded, it falls back to deterministic hash-derived vectors. This fallback permits the indexing code to continue, but it is not semantic embedding inference. Retrieval also includes keyword and evidence scoring. A combined score in the vector service weights dense score, keyword score, and evidence score; the code uses 0.45, 0.35, and 0.20 respectively in the scoring path.

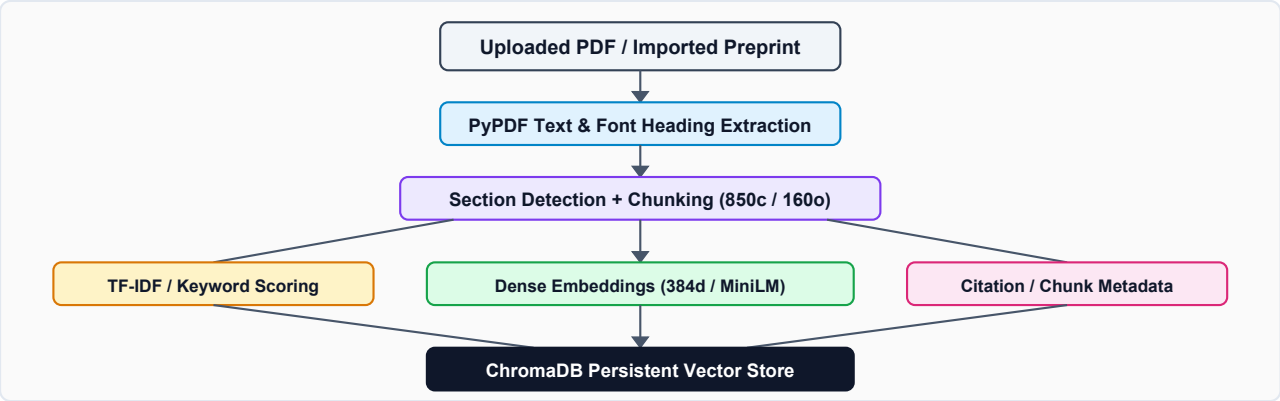


Figure 4.3: PDF processing, chunking, embedding, and RAG retrieval.

4.7 Hallucination Firewall Architecture

The LLM service contains an evidence-oriented response verification stage referred to in the implementation as a hallucination firewall. Its purpose is to reduce unsupported claims and citation mismatches by checking generated content against evidence and source metadata available to the response pipeline. It functions as a passive evidence auditor and accounting safeguard, not a mathematical proof of truth.

The firewall must be interpreted within the limits of the indexed corpus. If a paper is missing, a PDF is scanned, extraction is incomplete, or retrieval selects irrelevant chunks, the verification stage may not have sufficient evidence. A citation that points to a retrieved chunk can still be misinterpreted by the model. The system flags unsupported claims with an audit warning rather than attempting an autonomous text revision, making limitations transparent to the researcher.

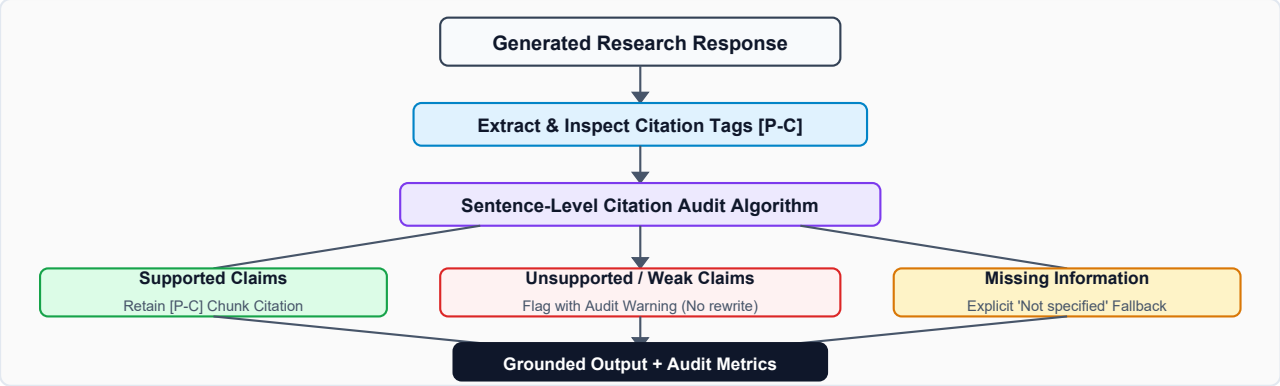


Figure 4.4: Evidence and citation verification concept.

4.8 Database Design

The relational model uses SQLAlchemy entities. The User entity stores account identity and password hash. A user owns multiple workspaces. A workspace contains papers, documents, and chat sessions. A chat session belongs to a user and a workspace and contains ordered chat messages. Paper records store bibliographic metadata, access URLs, local file path, extracted text, indexing status, and workspace association. Document records store generated outputs such as summaries, insights, literature reviews, comparisons, and reviews.

ChromaDB is separate from the relational database. It stores vectorized chunks and metadata used for retrieval. The SQLite database stores application entities and does not replace the vector index.

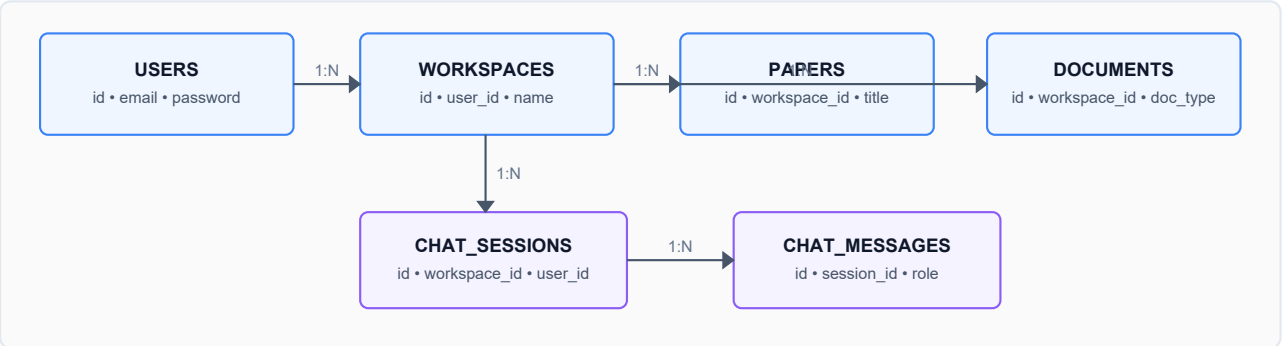


Figure 4.5: Simplified relational data model based on SQLAlchemy entities.

4.9 Data Flow Diagram – Level 0

At Level 0, the researcher is the primary external actor. The user sends a query, upload, or workspace action to ResearchPilot AI and receives answers, paper metadata, and execution traces. The system may communicate with academic sources for paper discovery and with an LLM provider for generation. Uploaded PDF content is processed within the application.

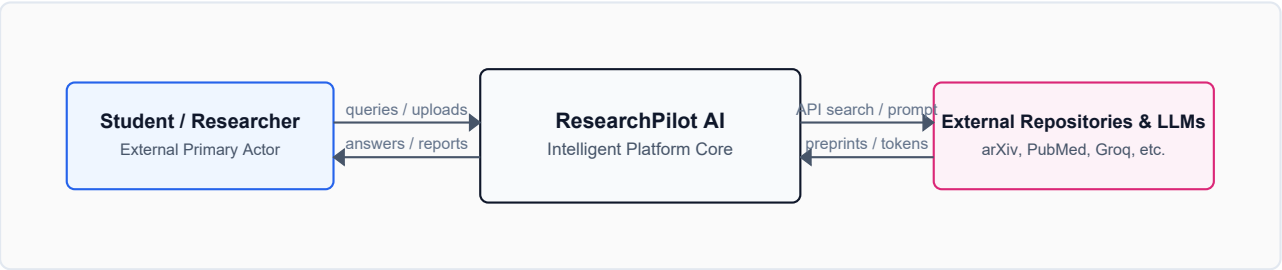


Figure 4.6: DFD Level 0 – Context diagram.

4.9 Data Flow Diagram – Level 1

At Level 1, the user request first passes through authentication and workspace-access checks. The relevant router or supervisor dispatches the request. Search requests use academic connectors; upload requests use PDF extraction and vector indexing; research questions use retrieval and analysis tools. The LLM service generates the requested content and the evidence-verification logic checks support before the response is returned.

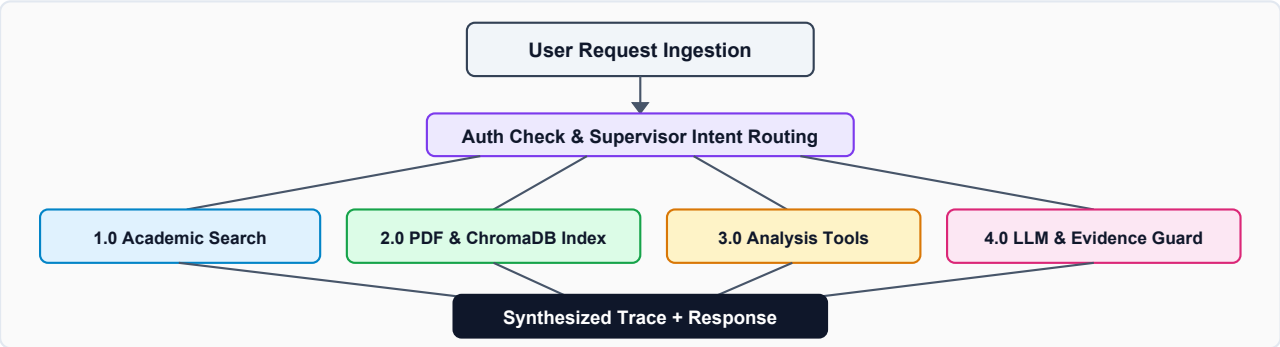


Figure 4.7: DFD Level 1 – Internal processing.

4.10 Sequence Diagram

The sequence diagram represents the supervisor-assisted chat flow. The browser submits a query to FastAPI, which validates user and workspace access before dispatching to the Supervisor Agent. The supervisor classifies intent and invokes the selected tool. The tool retrieves ChromaDB chunks or external metadata, requesting LLM generation and firewall verification where applicable. The grounded result returns through the supervisor, which prepends the decision trace banner. FastAPI persists the exchange in the SQLite database and returns the response to the browser.

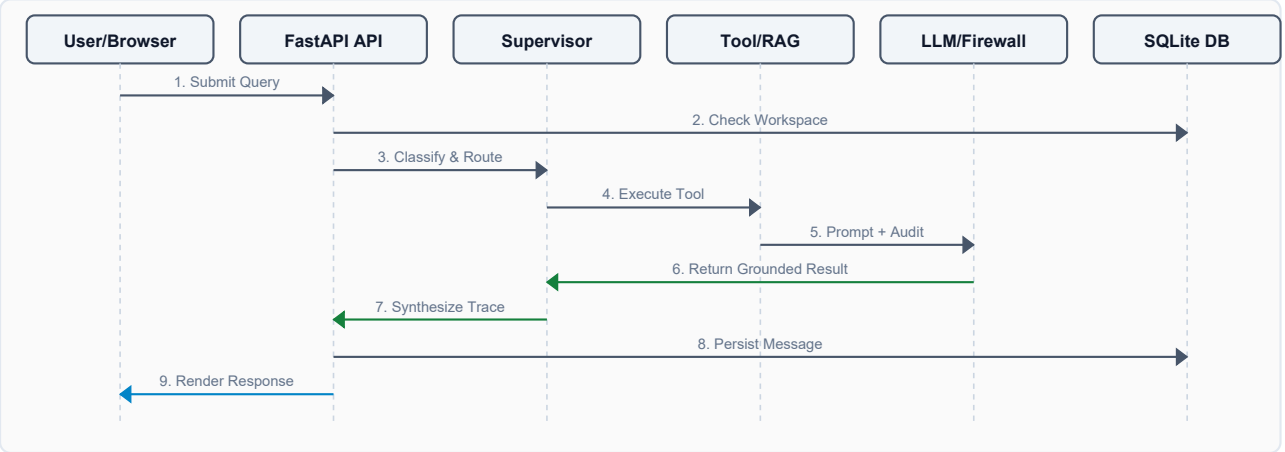


Figure 4.8: Sequence diagram for a supervisor-assisted research query.

4.11 Use Case Diagram

The primary actor is the student or researcher. The user can manage an account, create or select a workspace, search and import papers, upload PDFs, chat with workspace literature, and run structured research analysis. Academic indexes are external systems used during paper discovery.

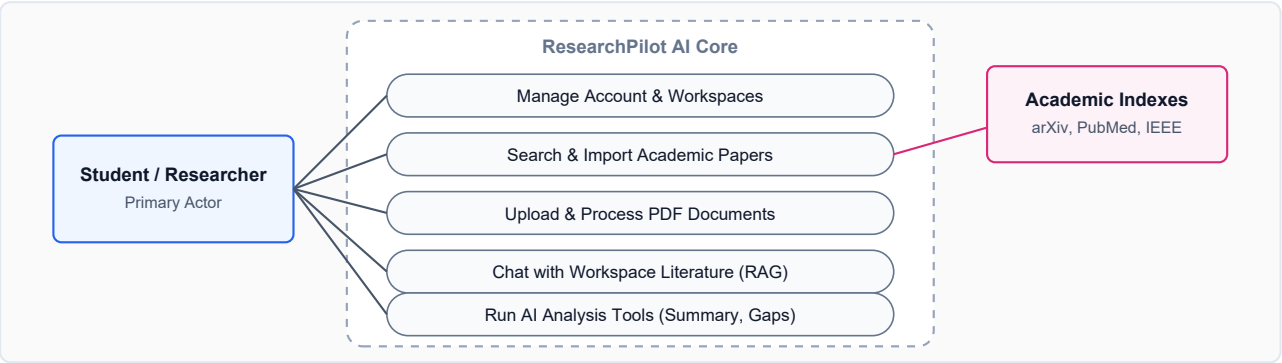


Figure 4.9: Use case view of ResearchPilot AI.

CHAPTER 5: IMPLEMENTATION

5.1 Frontend Implementation

The frontend is implemented using React 19, TypeScript, and Vite. React Router DOM provides page navigation. Axios is used as the API client, and the application includes authentication and chat contexts. The available pages include Home, Login, Register, Dashboard, Workspace, DocSpace, SearchPapers, UploadPDF, ResearchChat, and AITools. Tailwind CSS is present in the frontend dependencies and styling configuration; the project prompt permits CSS or Tailwind according to the existing implementation.

The frontend separates page-level workflows from reusable components such as the header, sidebar, protected route, markdown renderer, and floating assistant. The authentication context supports user session state and protected navigation. Chat context coordinates chat state and message submission.

5.2 Backend Implementation

The backend uses FastAPI and Uvicorn. Application startup registers routers for authentication, workspaces, papers, chat, assistant, tools, and system operations. SQLAlchemy provides relational persistence and dependency-managed database sessions. Pydantic schemas validate request and response structures. The service layer contains the academic source connectors, hybrid search, vector indexing, LLM provider calls, supervisor routing, and evidence guard logic.

Authentication uses password hashing and JWT-based access control. Protected endpoints obtain the current user and check workspace ownership before accessing workspace papers or documents. Environment configuration is loaded through Pydantic settings. Secrets and provider keys should be supplied through environment variables rather than embedded in source code.

5.3 Database and Workspace Management

The User, Workspace, Paper, Document, ChatSession, and ChatMessage models form the principal relational data model. A workspace is owned by one user and can contain multiple papers, generated documents, and chat sessions. Workspace scoping is important because it prevents a research query from silently mixing documents from unrelated research projects.

Chat sessions preserve message records in the relational database. The assistant endpoint can also accept a bounded recent history in the request. These mechanisms provide conversation context for an active research interaction; they should not be described as autonomous long-term memory or a self-updating knowledge graph.

5.4 Academic Paper Search Implementation

The HybridSearchService coordinates several source adapters concurrently. The source code queries nine initial discovery providers: API-backed services (arXiv, PubMed, Semantic Scholar, Crossref, CORE, DOAJ) and web scrapers (IEEE, ACM, and conference proceedings for NeurIPS, ICML, and ICLR). OpenAlex is integrated as a secondary resolver during access-URL enrichment rather than an initial discovery connector. It gathers results, tolerates individual provider exceptions, filters to approved sources, deduplicates records, enriches access URLs, ranks results, and diversifies the final list.

Source coverage depends on live service availability, rate limits, access rules, and connector configuration. Although the user-provided project brief mentions other academic sources, only the connectors present in the uploaded repository are described here. Crossref and PubMed are confirmed by the service modules; Semantic Scholar and arXiv are also present.

5.5 PDF Processing and Vector Storage

The paper upload endpoint accepts PDF files, stores them in the configured upload directory, and extracts text with PyPDF. If no text can be extracted, the endpoint returns an error indicating that the file may be scanned or image-only. A Paper record is created with the extracted text and local file path, then the vector service indexes the content.

The vector service detects common section labels including abstract, introduction, methodology, results, limitations, future work, and references. It uses paragraph-aware chunking with a default size of 850 characters and overlap of 160 characters. Chunk metadata supports later retrieval and source attribution. ChromaDB persists the research_papers collection under the configured data directory.

5.6 RAG-Based Research Chat

For a normal workspace question, the vector service searches the active workspace and returns up to eight chunks. If specific paper identifiers are selected, the service retrieves chunks from those papers instead. The LLM generation function receives the user question, retrieved content, chat history, and active paper metadata. The assistant API response includes a grounded flag and source descriptors containing paper ID, title, chunk index, and section where available.

The assistant endpoint also recognises broad-context requests such as 'summarize my uploaded papers', 'compare papers', or 'literature review' and can assemble a broader set of workspace chunks. The chat experience should be understood as retrieval-grounded generation rather than a guarantee that every generated sentence is fully supported.

5.7 Supervisor Agent Implementation

The ResearchSupervisorAgent class defines the six tool identifiers and metadata. Its run method obtains workspace papers, narrows them to selected paper IDs when supplied, classifies the user's intent, and invokes the matching execution method. The response is prefixed with a supervisor banner showing intent, tool, reasoning, and execution status.

The supervisor is a bounded controller. It does not generate an open-ended research plan, create new tools, or delegate work to independent autonomous agents. It selects one existing tool for the current request. The tool itself may perform retrieval and call the LLM service, but this does not establish multi-agent communication.

5.8 Intent Classification and Tool Routing

The classifier normalises user queries to lowercase and applies ordered regular-expression pattern matching. The execution order is deterministic: (1) Paper Comparison, (2) Literature Review, (3) Paper Summarization, (4) Research Gap Identification, (5) Academic Paper Search, and (6) Workspace RAG Q&A.;

A critical design consideration is intent disambiguation when queries mix multiple keywords. For instance, a query such as 'Summarize the methodology, dataset, and limitations of paper X' contains both the verb 'summarize' and the keyword 'limitations'. Under the supervisor's priority rules, explicit summarization directives take precedence, preventing the query from being misrouted to Research Gap Identification. Conversely, Research Gap Identification requires explicit exploratory directives (e.g., identify, detect, uncover, unaddressed). Similarly, queries containing phrases like 'papers on deep learning' will not trigger external academic search if the user explicitly asked to 'identify gaps' or 'compare'. If a query mentions a specific paper, _extract_target_paper_title() resolves the paper record using quoted titles, exact substrings, normalized alphanumeric tokens, or stopword-filtered set overlap before tool invocation.

5.9 AI Research Tools Implementation

Academic Paper Search calls the hybrid search service and formats preprint titles, authors, venues, publication dates, and access URLs directly into Markdown without invoking the LLM or firewall. Paper Summarization extracts up to eight relevant chunks for the resolved paper and requests a strictly structured summary containing five mandatory sections: ### Objective, ### Methodology, ### Dataset, ### Key Findings, and ### Limitations. Every factual assertion must append an inline evidence citation tag formatted as [P{paper_id}-C{chunk_index} | {Title}]. If retrieved chunks lack information for a given heading, the system explicitly outputs 'Not specified in the available paper content.' rather than generating speculative filler. Literature Review prepares evidence from up to six workspace papers and requests a thematic synthesis. Paper Comparison prepares up to five papers and constructs a side-by-side Markdown comparison matrix. Research Gap Identification analyzes methodology and limitation chunks across up to six workspace papers, categorizing open challenges with grounded paper citations.

5.10 Hallucination Firewall

The LLM service implements a post-generation verification component termed the Automated Hallucination Firewall. Following text generation, _extract_hallucination_stats() segments the report into individual sentences and audits them against retrieved evidence citations. Sentences bearing explicit chunk citations ([P\d+-C\d+]) are classified as Directly Supported; sentences containing cautious phrasing (may, might, suggests) are classified as Inferred; and ungrounded statements are classified as Needs Review or Unsupported.

The firewall calculates the exact percentage of paper-grounded claims and appends an audit block to the response. If unsupported statements exceed 10%, a visible warning banner is attached advising the user to inspect the source material. The firewall acts as an inspectable evidence-accounting mechanism; it does not autonomously rewrite generated text or guarantee absolute external factual truth.

5.11 User Interface and Screenshots

The uploaded repository contains frontend page components but does not include a complete set of real application screenshots suitable for insertion into the report. The following figure spaces are therefore intentionally marked as screenshot placeholders. Replace each placeholder with a screenshot captured from the running project before final submission. No fabricated UI images are presented as actual application evidence.

- Figure 5.1: Login page – [INSERT ACTUAL SCREENSHOT]
- Figure 5.2: Dashboard – [INSERT ACTUAL SCREENSHOT]
- Figure 5.3: Workspace management – [INSERT ACTUAL SCREENSHOT]
- Figure 5.4: Academic paper search – [INSERT ACTUAL SCREENSHOT]
- Figure 5.5: PDF upload – [INSERT ACTUAL SCREENSHOT]
- Figure 5.6: Research Chat – [INSERT ACTUAL SCREENSHOT]
- Figure 5.7: Supervisor Agent execution trace – [INSERT ACTUAL SCREENSHOT]
- Figure 5.8: Paper summarization – [INSERT ACTUAL SCREENSHOT]
- Figure 5.9: Research gap analysis – [INSERT ACTUAL SCREENSHOT]
- Figure 5.10: Literature review – [INSERT ACTUAL SCREENSHOT]
- Figure 5.11: Paper comparison – [INSERT ACTUAL SCREENSHOT]

CHAPTER 6: TESTING AND RESULTS

6.1 Introduction

Testing is required to verify authentication, workspace isolation, paper discovery, PDF extraction, vector indexing, retrieval, research-tool routing, LLM generation, and evidence checking. The repository includes test scripts named test_supervisor.py and test_agent_execution.py. These scripts define intent-classification checks and tool-execution scenarios. The presence of test scripts is not, by itself, proof that the tests were executed successfully in the environment used to prepare this report.

This report therefore distinguishes test scenarios visible in the source code from actual execution evidence. No new test run, performance measurement, accuracy score, or deployment validation is claimed here.

6.2 Test Cases

Test ID	Input / Action	Expected output	Actual result / status
TC01	Register with valid account details	Account created and authenticated	Not independently executed – To be verified
TC02	Login with valid credentials	Access token/session returned	Not independently executed – To be verified
TC03	Access a workspace owned by another user	Access denied	To be verified
TC04	Create and select a workspace	Workspace appears in workspace list	To be verified
TC05	Search academic papers by topic	Relevant paper metadata and source links displayed	To be verified
TC06	Upload a text-based PDF	Text extracted and paper record created	To be verified
TC07	Upload scanned/image-only PDF	Clear extraction failure message	Expected behaviour from route; verify
TC08	Ask a factual question about indexed papers	Retrieved context and grounded answer returned	To be verified
TC09	Request a paper summary with no papers in workspace	Prompt to upload or import a paper	Defined in source; execution not independently verified
TC10	Request literature review with fewer than two papers	Insufficient-paper message displayed	Defined in source; execution not independently verified
TC11	Ask to compare two papers	Paper Comparison tool selected	Test script includes scenario; execution not independently verified
TC12	Ask to identify research gaps	Research Gap Identification selected	Test script includes scenario; execution not independently verified
TC13	Ask to search for recent papers	Academic Paper Search selected	Test script includes scenario; execution not independently verified
TC14	Ask a generic paper question	Workspace RAG Q&A; selected	Test script includes scenario; execution not independently verified
TC15	Make a non-academic request	Research-scope guard rejects obvious out-of-scope request	To be verified
TC16	Simulate unavailable LLM provider	Suitable provider-unavailable message returned	To be verified
TC17	Inspect supervisor trace	Intent, tool, reasoning, and status are visible	Trace format present in source; UI verification pending
TC18	Check generated citations against retrieved passages	Unsupported citation/claim is flagged or qualified	To be verified

6.3 Functional Results

The source code demonstrates that the application has dedicated modules for authentication, workspaces, academic paper discovery, PDF upload, vector indexing, research chat, and structured analysis. It also defines a supervisor execution trace and evidence-oriented response checks. These are implementation observations derived from the repository and should not be confused with independently measured runtime results.

Before submission, the team should execute each test case and replace the status column with observed outputs. Screenshots of successful workflows, API responses, and test logs should be added where appropriate.

6.4 Research Tool Outputs

The expected output of Academic Paper Search is a list of paper records with bibliographic details and access links. Paper Summarization is expected to describe objectives, methodology, dataset, key findings, and limitations. Literature Review is expected to organise common themes and disagreements across the available papers. Paper Comparison is expected to compare selected publications across dimensions such as method, datasets, results, and limitations. Research Gap Identification is expected to derive candidate gaps from the workspace literature. Workspace RAG Q&A; is expected to answer a question using retrieved chunks and available source metadata.

These descriptions represent intended tool behaviour from the code structure and prompts. Actual output quality must be evaluated using representative research questions and human review.

6.5 Supervisor Agent Execution Results

The test_agent_execution.py file contains scenarios for an empty workspace, a populated workspace, academic search, summarisation, comparison, and literature review. It asserts that the expected tool label appears in the result and that empty-workspace gap analysis asks the user to upload or import literature instead of fabricating gaps. test_supervisor.py contains a set of intent-routing examples for checking the selected tool.

Because this report preparation did not execute those scripts against a verified configured runtime, the results are recorded as 'source test scenario present; actual run not recorded'. The team should run the scripts and attach the terminal output before treating the tests as passed.

6.6 Hallucination Firewall Results

The code contains a hallucination-firewall pathway in the LLM service. The expected evaluation is to supply responses with supported claims, unsupported claims, incorrect citations, and insufficient retrieved context. The team should record whether the guard preserves supported claims, flags unsupported statements with warnings and audit statistics, and reports evidence gaps. No numerical firewall accuracy or success rate is reported because no independently executed evaluation log was supplied.

6.7 Limitations and Observations

- PDF text extraction depends on selectable text; scanned documents may require OCR, which is not established as part of the described upload pipeline.
- Semantic retrieval quality depends on loading the Sentence Transformers model. The deterministic mock-vector fallback is not a semantic embedding substitute.
- Academic search results depend on external source availability, access restrictions, and connector behaviour.
- LLM output depends on provider configuration, rate limits, and network availability.
- Pattern-based intent classification can misroute ambiguous multi-intent queries.
- Evidence checks reduce risk but cannot guarantee factual correctness or complete citation validity.
- No measured response time, accuracy, task-success rate, or user-satisfaction score is included.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

ResearchPilot AI is an academic research assistant designed to support literature discovery, workspace organisation, PDF processing, retrieval-grounded question answering, and structured research analysis. The application combines a React and TypeScript frontend with a FastAPI backend, SQLite and SQLAlchemy relational persistence, ChromaDB vector storage, PyPDF extraction, and a configurable LLM provider layer.

The Supervisor Agent is the central routing component. It identifies the user's research intention using an ordered set of classification patterns and invokes one of six specialised research tools. The execution trace makes the routing decision visible by reporting the intent, selected tool, rationale, and execution status. RAG provides document-grounded context for workspace questions, while the Hallucination Firewall provides a separate evidence-oriented check on generated claims and citations.

The system is accurately characterised as a partially agentic AI application with a Supervisor Agent and specialised research tools. It is not a fully autonomous multi-agent system. Its practical usefulness depends on the completeness of the workspace literature, PDF extraction, retrieval quality, provider availability, and the reliability of evidence verification.

7.2 Limitations

- Academic source connectors may be affected by external service changes, rate limits, or access restrictions.
- PDF extraction may fail for scanned or image-only files without OCR.
- The embedding fallback uses deterministic mock vectors when Sentence Transformers is unavailable; this does not provide semantic similarity.
- Intent routing is pattern-based and may not resolve complex or ambiguous requests perfectly.
- LLM generation can contain unsupported or incorrectly interpreted statements even when evidence is retrieved.
- The Hallucination Firewall cannot prove truth or verify claims beyond the accessible evidence corpus.
- Provider fallback behaviour depends on environment configuration, including any Groq-only restriction.
- Runtime test results and measured performance values were not supplied with the source archive.

7.3 Future Scope

- Introduce multi-step research planning with explicit user approval and inspectable intermediate steps.
- Extend orchestration to support more advanced agent coordination while preserving clear tool boundaries.
- Improve semantic retrieval with validated embeddings, reranking, query expansion, and retrieval evaluation.
- Add OCR support for scanned PDFs and improved document layout extraction.
- Develop richer persistent research memory for project notes, user annotations, and research decisions.
- Expand academic-source coverage and improve connector resilience and metadata normalisation.
- Strengthen evidence verification with claim-level provenance, citation resolution, and independent evaluation datasets.
- Add paper recommendation based on workspace topics, citation relationships, and research interests.
- Provide citation exports in BibTeX, APA, MLA, and Chicago formats.
- Add collaborative workspaces and shared research collections.
- Add visual knowledge maps linking papers, methods, datasets, and research themes.

CHAPTER 8: REFERENCES

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems 33. https://arxiv.org/abs/2005.11401
2. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://arxiv.org/abs/2210.03629
3. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. https://arxiv.org/abs/2302.04761
4. Karpukhin, V., Oğuz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. https://arxiv.org/abs/2004.04906
5. Manakul, P., Liusie, A., & Gales, M. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023. https://arxiv.org/abs/2303.08896
6. Huang, L., Yu, W., Ma, W., et al. (2023). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. https://arxiv.org/abs/2311.05232
7. Groq. Groq API Documentation. https://console.groq.com/docs
8. Meta AI. (2024). Llama 3.3 Model Card and Prompt Format. Meta AI Documentation. https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/
9. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. https://arxiv.org/abs/1908.10084
10. FastAPI. FastAPI Documentation. https://fastapi.tiangolo.com/
11. React. React Documentation. https://react.dev/
12. Vite. Vite Documentation. https://vite.dev/guide/
13. SQLAlchemy. SQLAlchemy Documentation. https://docs.sqlalchemy.org/
14. Chroma. Chroma Documentation. https://docs.trychroma.com/
15. Hugging Face. Sentence Transformers Documentation. https://www.sbert.net/
16. PyPDF. PyPDF Documentation. https://pypdf.readthedocs.io/
17. arXiv. arXiv API User Manual. https://info.arxiv.org/help/api/user-manual.html
18. PubMed. NCBI E-utilities Documentation. https://www.ncbi.nlm.nih.gov/books/NBK25501/
19. Semantic Scholar. Semantic Scholar API Documentation. https://api.semanticscholar.org/api-docs/
20. Crossref. Crossref REST API Documentation. https://www.crossref.org/documentation/retrieve-metadata/rest-api/
21. OpenAlex. OpenAlex API Documentation. https://docs.openalex.org/
22. DOAJ. DOAJ API Documentation. https://doaj.org/api/v2/docs
23. Project source code supplied for this report: ResearchPilot-AI-main repository archive (backend and frontend implementation).

APPENDIX A: IMPLEMENTATION WORKFLOW AND API RESPONSIBILITIES

A.1 Main Backend Responsibility Map

Module / file group	Main responsibility
app/routers/auth.py	Registration, login, token validation, and current-user dependency.
app/routers/workspaces.py	Workspace creation, listing, and ownership-scoped workspace operations.
app/routers/papers.py	Paper upload, PDF extraction, indexing, paper access, listing, and deletion.
app/routers/chat.py	Workspace chat sessions and persisted chat messages.
app/routers/assistant.py	Workspace assistant endpoint, retrieval, chat context, and source metadata.
app/routers/tools.py	Dedicated endpoints for summary, insights, review, and other analysis operations.
app/services/hybrid_search_service.py	Coordination, deduplication, ranking, and diversification of academic search results.
app/services/vector_service.py	Chunking, embedding generation, ChromaDB indexing, and retrieval scoring.
app/services/supervisor_agent.py	Intent classification, single-tool routing, tool execution, and trace generation.
app/services/llm_service.py	Prompt-based generation and configured provider handling.
app/services/guard_service.py	Research-scope query screening.

A.2 Illustrative Request Processing Pseudocode

The following pseudocode summarises the source-backed control flow; it is explanatory and is not copied as a complete source implementation.

1. Receive user query and workspace/session identifiers
2. Validate the user and verify access to the workspace
3. Load papers belonging to the active workspace
4. Classify intent using the supervisor's ordered patterns (Comparison -> Review -> Summary -> Gaps -> Search -> RAG)
5. Select one registered research tool
6. Execute the selected tool:
   a. Search academic sources (Direct preprint metadata formatting, bypassing LLM), OR
   b. Build paper evidence and summarise/review/compare/analyse gaps via LLM, OR
   c. Retrieve workspace chunks and answer the question via RAG
7. Handle missing papers, provider failures, or tool exceptions
8. Apply evidence/citation verification to applicable LLM-generated responses. Academic Paper Search directly formats and returns discovery results without invoking the LLM or Hallucination Firewall.
9. Return composite answer with intent, selected tool, rationale, and status

APPENDIX B: APPLICATION SCREENSHOT PLACEHOLDERS

The boxes below are intentionally blank. Replace them with genuine screenshots captured from the running ResearchPilot AI application. Do not submit these placeholders as proof of execution.

Figure 5.1 – Login page

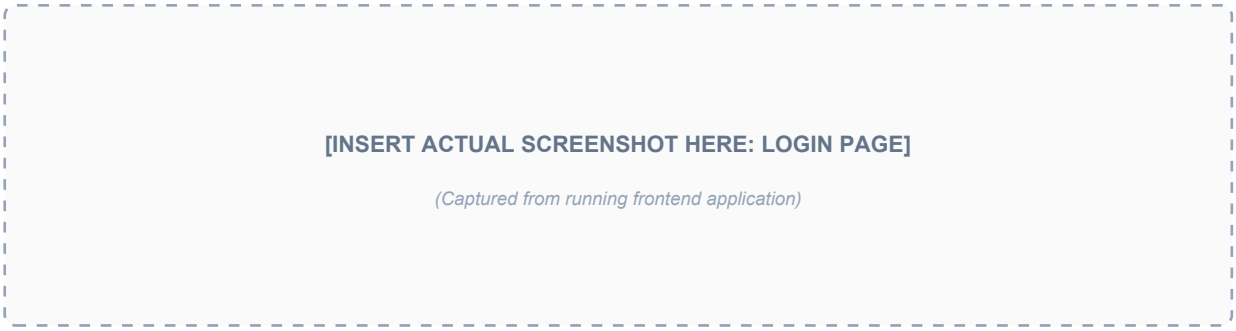


Figure 5.2 – Dashboard

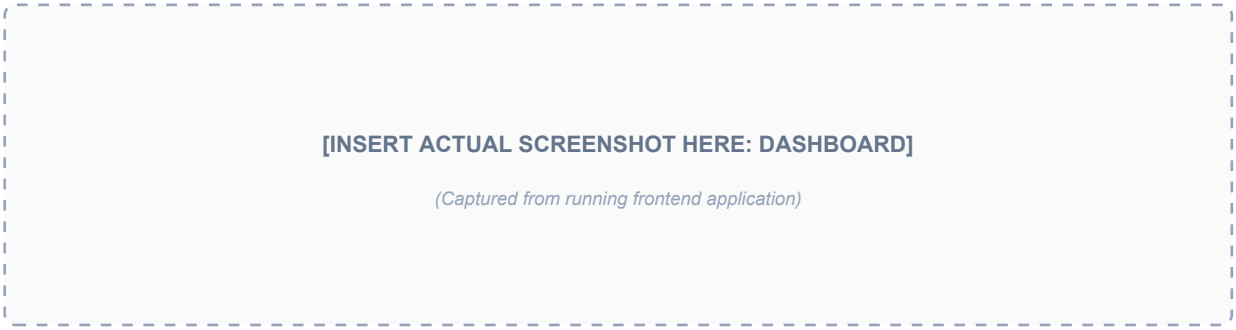


Figure 5.3 – Workspace management

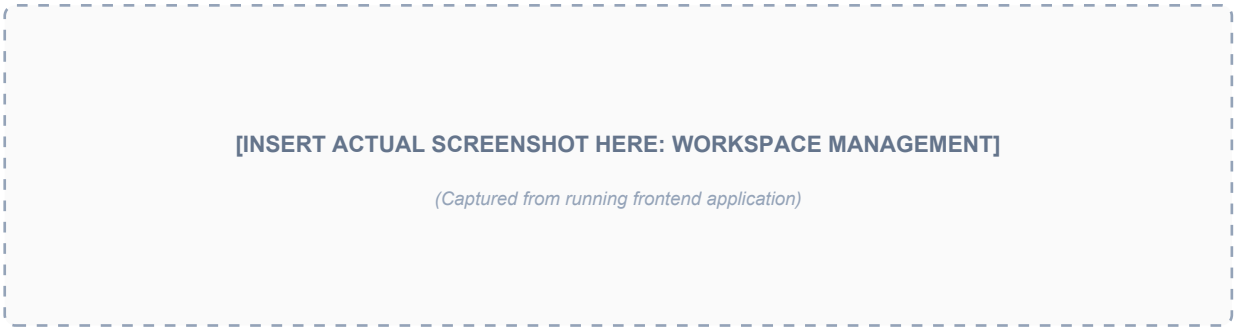
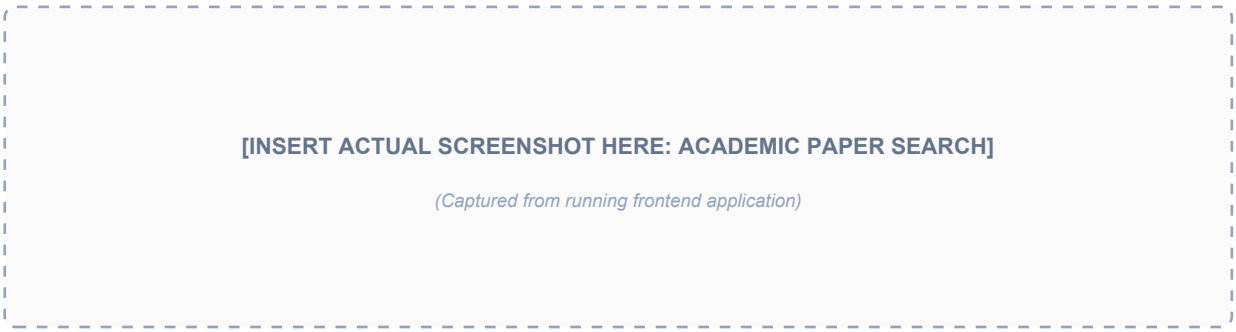


Figure 5.4 – Academic paper search



APPENDIX B: APPLICATION SCREENSHOT PLACEHOLDERS

The boxes below are intentionally blank. Replace them with genuine screenshots captured from the running ResearchPilot AI application. Do not submit these placeholders as proof of execution.

Figure 5.5 – PDF upload

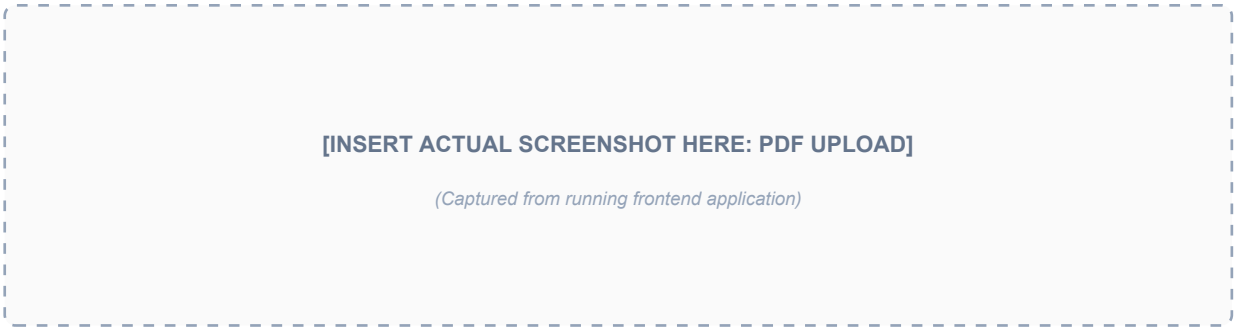


Figure 5.6 – Research Chat

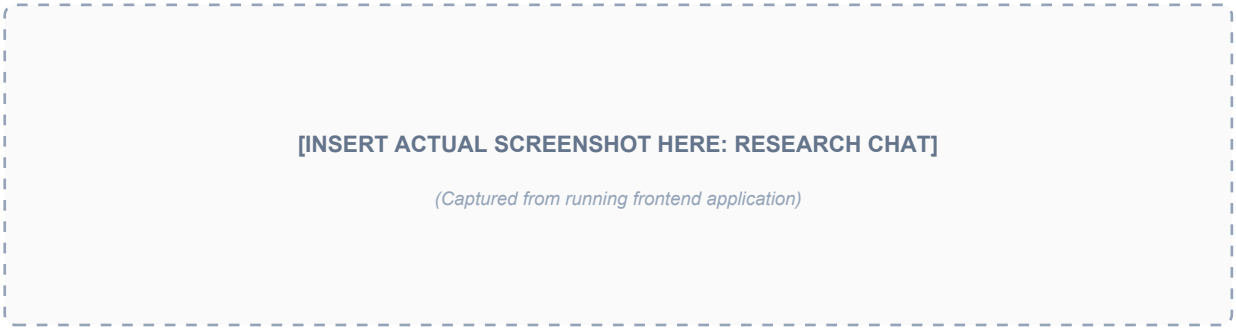


Figure 5.7 – Supervisor Agent execution trace

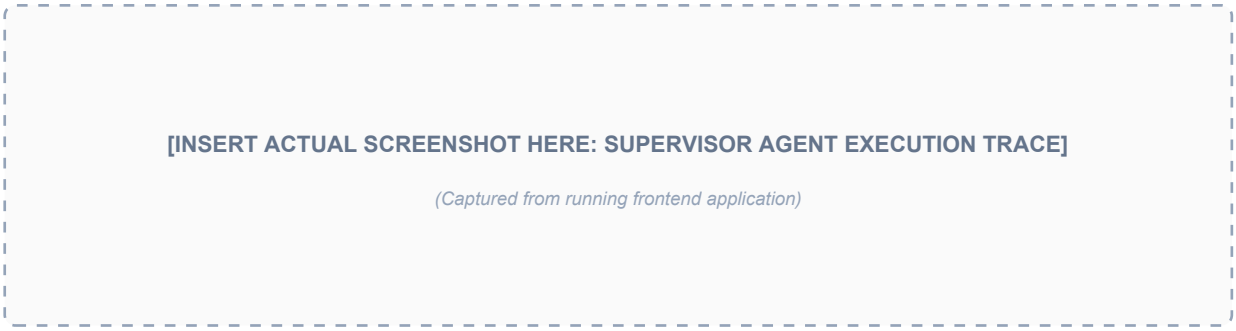
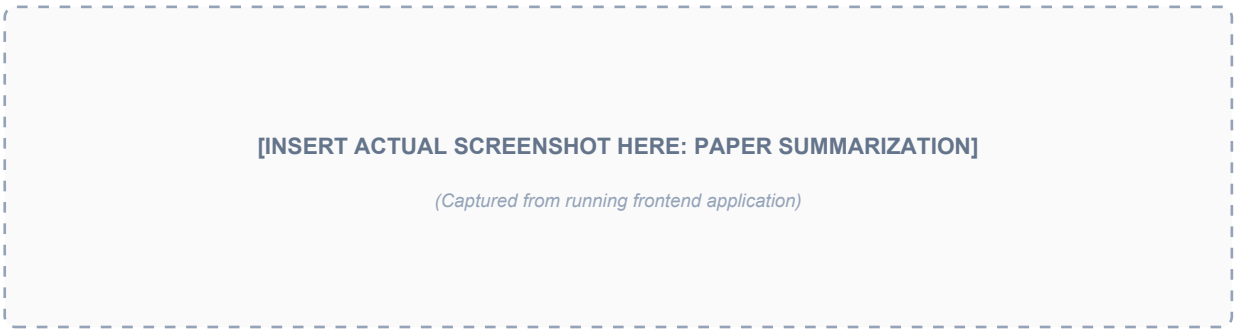


Figure 5.8 – Paper summarization



APPENDIX B: APPLICATION SCREENSHOT PLACEHOLDERS

The boxes below are intentionally blank. Replace them with genuine screenshots captured from the running ResearchPilot AI application. Do not submit these placeholders as proof of execution.

Figure 5.9 – Research gap analysis

[INSERT ACTUAL SCREENSHOT HERE: RESEARCH GAP ANALYSIS]

(Captured from running frontend application)

Figure 5.10 – Literature review

[INSERT ACTUAL SCREENSHOT HERE: LITERATURE REVIEW]

(Captured from running frontend application)

Figure 5.11 – Paper comparison

[INSERT ACTUAL SCREENSHOT HERE: PAPER COMPARISON]

(Captured from running frontend application)